

JVM, JRE & JDK Explained

- [JVM, JRE & JDK Explained](#)
 - [1. Introduction](#)
 - [2. Compiler vs. Interpreter](#)
 - [3. How the JVM Converts Bytecode → Machine Code](#)
 - [4. JVM's Three Jobs](#)
 - [5. The Hierarchy: JVM → JRE → JDK](#)
 - [6. Java Editions: JSE vs JEE vs JME](#)
 - [7. Running Your First Java Program](#)
 - [★ One-Page Revision](#)
 - [📝 Practice Questions](#)
 - [🔥 Quick FAQ](#)

JVM, JRE & JDK Explained

Writing Your First Java Program — Lecture 2

1. Introduction

Before writing real Java code, it's important to understand **what actually runs your program** behind the scenes — and the three terms that confuse almost every beginner: **JVM, JRE, and JDK**.

This lecture also compares **compilers vs. interpreters**, explains how Java uses both, and covers Java's three editions (**JSE, JEE, JME**) before running an actual "Hello World" program.

2. Compiler vs. Interpreter

Both compilers and interpreters do the same basic job: **convert source code into machine code**. The difference is *how*.

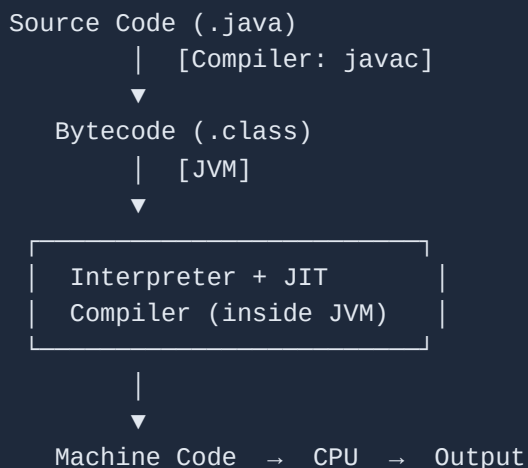
	Compiler	Interpreter
Converts	Entire code, all at once	Line by line
Speed to start running	Slower to start (waits for full conversion)	Starts running almost immediately
Used by	C, C++	Python (roughly)

💡 Easy Hinglish Explanation

"Compiler poora code ek baar mein padhta hai aur pura machine code bana deta hai. Interpreter ek line padhta hai, usko turant machine code mein convert karke run kar deta hai, phir agli line pe jaata hai."

★ **Exam Tip** Java is **neither** purely compiled nor purely interpreted — it's **both**. Source code → bytecode uses a **compiler**. Bytecode → machine code uses an **interpreter + JIT compiler** together.

3. How the JVM Converts Bytecode → Machine Code



Why not just use a compiler inside the JVM too? In the 1990s, hardware was slow — limited RAM, slow disks. Using an **interpreter** let Java programs start running almost instantly, line by line, instead of waiting for a full compile step.

Why add a JIT compiler later? As hardware got faster, Java added a **JIT (Just-In-Time) Compiler** — it doesn't replace the interpreter, it works *alongside* it:

- Code that runs **frequently** → compiled directly to machine code by the **JIT compiler** (fast)
- Code that runs **rarely** → converted **line by line** by the **interpreter**

 **Important** This hybrid model (interpreter + JIT) is *why* Java is a little slower than C++ (which compiles straight to machine code) — but the difference today is negligible thanks to modern hardware.

4. JVM's Three Jobs

Job	What It Does
1. Bytecode → Machine Code	Using interpreter + JIT compiler
2. Security (Sandboxing)	Runs bytecode in a restricted environment, so untrusted downloaded code can't freely delete files or install malware
3. Garbage Collection	Automatically frees memory that's no longer being used (covered in depth later)

Easy Hinglish Explanation

"JVM ek 'virtual computer' hai — matlab ek asli machine nahi, balki ek environment jo tumhare system ke andar exist karta hai. Bytecode directly tumhare system pe nahi, JVM ke andar run hota hai — isi wajah se security milti hai."

5. The Hierarchy: JVM → JRE → JDK

Java's three core terms build on top of each other, like **concentric circles**:



Layer	Full Form	Contains	Purpose
JVM	Java Virtual Machine	—	Runs bytecode, converts it to machine code
JRE	Java Runtime Environment	JVM + Class Libraries	Lets you run a Java program
JDK	Java Development Kit	JRE + Compiler + Debugger + Dev tools	Lets you write, compile, and run Java programs

What are "Class Libraries"? Built-in, ready-made code for common tasks — like printing to the screen or reading a file — that you don't write yourself. Similar to `#include <iostream>` in C++; in Java, you `import` these instead.

⚠ **Common Mistake** Having only the JVM isn't enough to run a program — it has no class libraries. And today, you can't even install JRE alone anymore; **you need the full JDK** to develop Java applications.

6. Java Editions: JSE vs JEE vs JME

Edition	Full Form	Used For	Status
JSE	Java Standard Edition (Core Java)	General-purpose programming — classes, methods, OOP basics	Foundation of everything
JEE	Java Enterprise Edition (<i>now called Jakarta EE</i>)	Web apps, enterprise systems (e.g. Spring Boot)	Actively used
JME	Java Micro Edition	Old mobile/embedded devices	Obsolete — replaced by Android

📌 **Important:** JME has been almost entirely replaced by **Android** development (Java/Kotlin), so it's no longer worth learning today.

7. Running Your First Java Program

Step-by-step workflow:

1. Write code in a file — e.g. `Demo.java` (must match the class name, extension always `.java`)
2. **Compile** it into bytecode:

```
javac Demo.java
```

This generates `Demo.class` — the bytecode file. (If you open it directly, it won't be readable — it's an intermediate format, not English and not raw machine code.)

3. **Run** the bytecode:

```
java Demo
```

This tells the JVM (inside JRE) to convert the bytecode to machine code and execute it — the CPU runs it and prints the output.

Code Example

```
public class Demo {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Output:

```
Hello World
```

✓ **Best Practice:** Don't worry yet about what `class`, `static`, `public void main`, or `String[] args` mean — that syntax is covered in later lectures. For now, just focus on the **compile** → **run** workflow.

★ One-Page Revision

Core Definitions

- **Compiler** = converts entire source code to machine code at once
- **Interpreter** = converts source code to machine code line by line
- **JIT Compiler** = "Just-In-Time" — compiles frequently-used bytecode directly to machine code for speed
- **JVM** = Java Virtual Machine; converts bytecode → machine code, provides security (sandbox) and garbage collection
- **JRE** = JVM + Class Libraries; needed to **run** Java programs
- **JDK** = JRE + Compiler + Debugger; needed to **write and compile** Java programs

- **Class Libraries** = built-in, reusable code (e.g. for printing, file reading)

Key Facts

- Java is both compiled (source → bytecode) **and** interpreted (bytecode → machine code)
- JVM uses **interpreter + JIT compiler together** — not one or the other
- `javac Demo.java` → compiles to `Demo.class`
- `java Demo` → runs the bytecode via the JVM
- JSE = core Java · JEE = enterprise/web (Jakarta EE) · JME = obsolete, replaced by Android



Practice Questions

Basic

1. What is the difference between a compiler and an interpreter?
2. Expand JVM, JRE, and JDK.
3. What command compiles a Java file, and what command runs it?

Medium 4. Why does Java use both an interpreter and a JIT compiler instead of just one? 5. What extra components does JDK have that JRE doesn't?

Interview 6. Is Java a compiled language or an interpreted language? Justify your answer. 7. How does the JVM provide security through the sandbox model?



Quick FAQ

Do I need JRE separately from JDK? No — modern JDK installations already include it.

Is Java slower than C++? Slightly, due to the extra bytecode step — but the JIT compiler makes the difference barely noticeable today.

Should I learn JME? No — it's obsolete, fully replaced by Android.